

p

p4

Project ID: 16233



instructions and plot added
LOUIS TYRRELL OLIPHANT authored 5 days ago

Name	Last commit	Last update
README.md	instructions and plot added	5 days ago
example_plot.pdf	instructions and plot added	5 days ago

[README.md](#)

author: John Shawger, Sunaina Krishnamoorthy

Administrivia

- **Due date:** October 24th, 2023 at 11:59pm.
- **Handing it in:**
 - Copy all of the modified xv6 files to ~cs537-1/handin/<cslogin>/P4/ .
 - Turn in a screenshot of your graphs to Canvas.
- Late submissions
 - Projects may be turned in up to 3 days late but you will receive a penalty of 10 percentage points for every day it is turned in late.
 - Slip days:
 - In case you need extra time on projects, you each will have 2 slip days for individual projects and 2 slip days for group projects (4 total slip days for the semester). After the due date we will make a copy of the handin directory for on time grading.
 - To use a slip day you will submit your files with an additional file `slipdays.txt` in your regular project handin directory. This file should include one thing only and that is a single number, which is the number of slip days you want to use (ie. 1 or 2). Each consecutive day we will make a copy of any directories which contain one of these slipdays.txt files.
 - After using up your slip days you can get up to 90% if turned in 1 day late, 80% for 2 days late, and 70% for 3 days late. After 3 days we won't accept submissions.
 - Any exception will need to be requested from the instructors.
 - Example slipdays.txt

1

- Some tests are provided at ~cs537-1/tests/P4. There is a README.md file in that directory which contains the instructions to run the tests. The tests are partially complete and you are encouraged to create more tests.
- Questions: We will be using Piazza for all questions.
- Collaboration: The assignment must be done by yourself. Copying code (from others) is considered cheating. [Read this](#) for more info on what is OK and what is not. Please help us all have a good semester by not doing this.
- This project is to be done on the [Linux lab machines](#), so you can learn more about programming in C on a typical UNIX-based platform (Linux). Your solution will be tested on these machines.

xv6 MLFQ Scheduler

You will implement a multi-level feedback queue scheduler with cpu decay for xv6 in this project. Process priorities will be set using their prior cpu usage (details below), and processes which have used less cpu recently will have higher priority. This is a different MLFQ scheduler than presented in OSTEP, but the principle of adjusting priority based on prior cpu usage is the same. You will also implement a `nice()` system call, which lowers the priority of a process. On every tick, the scheduler will schedule the runnable process with the highest priority, or round-robin amongst processes with the same priority.

Most of the code for the scheduler is quite localized and can be found in `proc.c` ; the associated header file, `proc.h` is also quite useful to examine. To change the scheduler, not much needs to be done; study its control flow and then try some small changes.

Objectives

- To implement a simple MLFQ scheduler
- To understand the xv6 scheduler
- To understand context switches in xv6
- To implement system calls which modify process state

Project details

MLFQ Scheduler

xv6 uses a simple round-robin scheduler. In this project, you will modify the scheduler to schedule processes according to process priority. Priority is function of recent cpu usage. The kernel should keep track of cpu usage, by counting the number of ticks the process has spent running, for each process. To reward processes which have not used the cpu recently, the total cpu usage for each process is "decayed" when priorities are recalculated. Our decay function is simply to divide cpu usage in half.

In our MLFQ scheduler, process priorities will be recalculated every second. We will use the following method to determine priority:

```
def decay(cpu):
    return cpu/2

cpu = decay(cpu)
priority = cpu/2 + nice
```

For new processes, priority, nice, and cpu values should all be set to zero. **Higher priority values correspond to lower process priority, lower values correspond to higher process priority.**

A note about xv6 timing

A tick in xv6 represents about 10ms of time. Thus, priorities should be recalculated every 100 ticks. In both the default xv6 scheduler and ours, the scheduler should be run on every tick. In other words, a high-priority process will yield() on every tick, but may be rescheduled if it is still the highest-priority process on the system.

Modify sleep() system call

The behavior of the xv6 sleep() system call interferes with our scheduler. sleep(n) puts a process to sleep for n ticks. On every timer interrupt, xv6 wakes up all sleeping processes. If the process still has more ticks to sleep, it goes back to sleep until the next timer interrupt when the process repeats.

You should modify the sleep() system call so that processes are only runnable after all of their sleeping ticks have expired.

New System Calls

You'll need two new system calls to implement this scheduler. The first is a nice() system call which allows a process to lower its priority, and the second is getschedstate() which returns some information about priorities and ticks to the user.

nice() system call

The nice() system call allows a process to voluntarily decrease its priority. The nice value is added to a process's priority. You should implement a system call with the following prototype

```
int nice(int n);
```

nice() should set the current process's nice value to n and return the previous nice value, or -1 on error. Nice values lower than 0 or greater than 20 are considered invalid and the system call should return an error if a user attempts to set a nice value outside these bounds.

getschedstate() system call

This routine (detailed below) returns relevant scheduler-related information about all running processes, including how many times each process has been chosen to run (ticks) and the process ID of each process. The structure pschedinfo is defined below; note, you cannot change this structure, and must use it exactly as is. This routine should return 0 if successful, and -1 otherwise (if, for example, a bad or NULL pointer is passed into the kernel).

```
int getschedstate(struct pschedinfo *);
```

You'll need to understand how to fill in the structure `pschedinfo` in the kernel and pass the results to user space. Please create a new file called `psched.h` and copy the contents from below exactly. You will need to include this header file in your code. Please do not modify the structure of this `pschedinfo` struct.

```
#ifndef _PSCHED_H_
#define _PSCHED_H_

#include "param.h"

struct pschedinfo {
    int inuse[NPROC];    // whether this slot of the process table is in use (1 or 0)
    int priority[NPROC]; // the priority of each process
    int nice[NPROC];     // the nice value of each process
    int pid[NPROC];      // the PID of each process
    int ticks[NPROC];    // the number of ticks each process has accumulated
};

#endif // _PSCHED_H_
```

Good examples of how to pass arguments into the kernel are found in existing system calls. In particular, follow the path of `read()`, which will lead you to `sys_read()`, which will show you how to use `argptr()` (and related calls) to obtain a pointer that has been passed into the kernel. Note how careful the kernel is with pointers passed from user space -- they are a security threat(!), and thus must be checked very carefully before usage.

Makefile **changes**

In order for our tests to work, we require two changes to the `xv6 Makefile`. Ensure that `CPUS := 1`, as we are only considering scheduling on a single core. Finally, disable compiler optimizations by changing the `CFLAGS` from `-O2` to `-O0`. Some of the tests use spin loops, which may be removed by the compiler if optimizations are turned on.

Evaluation

Beyond the usual code, you will also have to compare and evaluate your brand new scheduler with the original `xv6` scheduler, and to do so, we would like you to run `test13` (provided) with your new code and with the original code, and plot a graph based on the results.

- Create one graph for original `xv6` scheduler, and one graph for your `mlfq` scheduler. Each graph must show a plot of Total ticks (Y axis) vs Time (X axis), showing the execution of the processes in `test13` in different colors.
- Note: You will need to modify scheduler code in the kernel to print out information that you need to plot the graphs. Turn in a version without the print statements (see below).
- The graph can be generated using plotting software or be hand-drawn clearly showing the point co-ordinates. Submit a screenshot or an image of the graph to **Canvas** under Project 4.
- Submission: Please upload screenshots of your graphs to Canvas (NOT the handin directory). In your handin directory, you only need to copy the files of the MODIFIED MLFQ scheduler, not the original `xv6`. Please turn in a version of your code without logging enabled, as it can interfere with some of the performance tests.

Hints

- You will need to keep track of process priorities and cpu usage. Where should these values be stored? Where in the process of context switching should they be updated?
- Most of the scheduler code can be found in `proc.c`
- Review what happens on a timer interrupt in `trap.c`
- Ensure that `xv6` is running with a single cpu. In the `Makefile`, `CPUS :=1`
- Run `xv6` using `make qemu-nox`. Exit `xv6` using `Ctrl-a x`.

Futher Reading

Please take a look at these references for more information on the concepts, and watch the discussion video to see Prof. Remzi's explanation of the `xv6` scheduler.

- ["An Analysis of Decay-Usage Scheduling in Multiprocessors" by D.H.J. Epema. SIGMETRICS '95.](#)
- The Design of the UNIX Operating System, Maurice J. Bach (Chapter 8). Our scheduler is similar to the UNIX System V scheduler described in this book.
- [Remzi's discussion video](#). A look at context switches and scheduling in `xv6`.

